

Manual Técnico

SIOL-SAVA — Backend de Gestión y Trazabilidad de Entregas de Última Milla

Proyecto	SIOL-SAVA (MVP de tesis)
Componente	API Backend (FastAPI)
Versión	1.0.0
Fecha	Junio 2026
Stack	Python 3.11 · FastAPI · PostgreSQL 15 · SQLAlchemy · Alembic · Docker
Arquitectura	Clean Architecture (api → service → repository → models)

Índice

- 1. Introducción y stack tecnológico
- 2. Arquitectura limpia (cómo se conecta todo)
- 3. Puesta en marcha (Docker, variables de entorno)
- 4. Conceptos clave (códigos legibles, roles, estados)
- 5. Autenticación: cómo obtener y usar el token
- 6. Catálogo de APIs (módulo por módulo)
- 7. Flujo completo de extremo a extremo
- 8. Seguridad (OWASP, secretos, CI/CD)
- 9. Apéndice: mapa de archivos del proyecto

1. Introducción y stack tecnológico

SIOL-SAVA es un sistema de gestión y trazabilidad de entregas logísticas de última milla. Este backend expone una API REST que consumen dos clientes: el **panel web del administrador** (carga de pedidos, armado de rutas, monitoreo) y la **app móvil del conductor** (descarga de ruta, validación de paquetes, registro de entregas con evidencia).

El backend cubre 4 fases funcionales:

- **Fase 1 — Autenticación:** usuarios, roles y tokens JWT.
- **Fase 2 — Inbound y motor logístico:** carga de pedidos por Excel, geocodificación, zonas y armado/optimización de rutas (VRP).
- **Fase 3 — Operaciones de última milla:** endpoints de la app móvil del conductor.
- **Fase 4 — Trazabilidad y CI/CD:** dashboard, historial de paquetes y automatización.

Tecnologías:

Lenguaje	Python 3.11	Lógica del servidor
Framework API	FastAPI	Endpoints REST + documentación Swagger automática
Base de datos	PostgreSQL 15	Almacenamiento de datos
ORM	SQLAlchemy	Acceso a datos sin SQL manual (evita inyección)
Migraciones	Alembic	Control de versiones del esquema de la BD
Geocodificación	Geopy (Nominatim)	Convertir direcciones en coordenadas
Excel	Pandas + openpyxl	Leer la carga masiva de pedidos
Contenedores	Docker + Docker Compose	Levantar API y BD con un comando
Seguridad	JWT (python-jose) + Argon2 (passlib)	Tokens y cifrado de contraseñas

2. Arquitectura limpia (cómo se conecta todo)

El código está organizado en **capas**. Cada petición fluye de arriba hacia abajo, y cada capa solo habla con la siguiente. Esto mantiene el código ordenado, testeable y fácil de mantener.

API / Router	app/api/	Recibe la petición HTTP, valida permisos (rol) y delega. No tiene lógica.
Servicio	app/services/	La lógica de negocio (reglas, validaciones, orquestación).
Repositorio	app/repositories/	Único lugar que consulta/escrbe en la base de datos.
Modelo	app/models/	Definición de las tablas (SQLAlchemy).
Schema	app/schemas/	Moldes de datos de entrada/salida (Pydantic) que validan tipos.
Core	app/core/	Seguridad (JWT, hash), configuración y códigos legibles.
DB	app/db/	Conexión y sesión de la base de datos.

Flujo de una petición (ejemplo: consultar la ruta activa):

```

Cliente (Swagger / App)
  | GET /api/conductor/ruta-activa (con token JWT)
  v
api/conductor.py      -> verifica rol 'conductor' (deps.py)
  v
services/ruta_service -> aplica la lógica de negocio
  v
repositories/ruta_repository -> consulta la BD
  v
models/ruta.py (tabla 'rutas') -> PostgreSQL
  ^ la respuesta sube por las mismas capas (validada por schemas)

```

3. Puesta en marcha

Requisitos: tener **Docker Desktop** instalado y corriendo.

Paso 1 — Variables de entorno:

Copia la plantilla y ajusta los valores (la clave secreta, credenciales, etc.):

```
copy .env.example .env      (Windows)
cp .env.example .env        (Linux/Mac)
```

Paso 2 — Levantar todo (API + base de datos):

```
docker compose down -v && docker compose up --build
```

El **-v** recrea la base de datos desde cero (necesario tras cambios de esquema). Al arrancar se crean las tablas y un **admin inicial** automáticamente.

Paso 3 — Abrir la documentación interactiva (Swagger):

```
http://localhost:8000/docs
```

Desde Swagger puedes probar TODOS los endpoints con el botón «Try it out». El endpoint de salud **GET /** confirma que el backend está vivo.

4. Conceptos clave

Códigos legibles

Cada registro tiene un **id** numérico interno (para las relaciones) y un **codigo** legible para identificarlo a simple vista:

usuarios (admin)	AD-NNN	AD-001
usuarios (conductor)	CO-NNN	CO-002
clientes_corporativos	CL-NNN	CL-001
vehiculos	VE-NNN	VE-001
pedidos	PD-NNN	PD-001 (es el tracking / lo que va en el QR)
rutas	RT-NNN	RT-001
ruta_detalle	RD-NNN	RD-001
historial_pedidos	HP-NNN	HP-001

Roles de usuario

- **admin**: panel web. Gestiona clientes, vehículos, pedidos, rutas y monitoreo.
- **conductor**: app móvil. Solo opera SU ruta del día.

Estados (máquina de estados)

Pedido	PENDIENTE → ASIGNADO → EN_RUTA → ENTREGADO / FALLIDO (o GEOCODIFICACION_FALLIDA)
Ruta	CREADA → EN_PROGRESO → FINALIZADA
Entrega (detalle)	PENDIENTE → ENTREGADO / FALLIDO

5. Autenticación: cómo obtener y usar el token

Casi todos los endpoints están protegidos. Para usarlos necesitas un **token JWT**, que se obtiene iniciando sesión. El token se envía en la cabecera `Authorization: Bearer <token>`.

En Swagger es automático:

- Pulsa el botón verde «Authorize» (arriba a la derecha).
- Escribe el correo en el campo «username» y la contraseña. Deja vacíos `client_id/secret`.
- A partir de ahí Swagger manda el token en cada petición. El candado pasa de abierto a cerrado.
- El token dura 8 horas; si caduca (error 401), vuelve a autorizar.
- Swagger guarda UN token: para cambiar de admin a conductor haz Logout y entra de nuevo.

Admin inicial (para el primer ingreso):

Al arrancar se crea automáticamente un administrador con las credenciales definidas en el `.env` (por defecto **admin@siol.com / admin123**). Con él inicias sesión y das de alta al resto de usuarios.

6. Catálogo de APIs (módulo por módulo)

Para cada endpoint se indica: rol requerido, para qué sirve, los datos de entrada, la respuesta y con qué partes del sistema se conecta.

6.1 Autenticación (/api/auth)

POST	/api/auth/login
Rol requerido	Público (sin token)
Para qué sirve	Inicia sesión y entrega el token JWT que usarán el resto de endpoints.
Entrada (inputs)	Formulario (x-www-form-urlencoded): username = correo password = contraseña
Respuesta	{ "access_token": "eyJhbGciOi...", "token_type": "bearer" }
Se conecta con	usuario_service → usuario_repository → tabla usuarios; core/security (verifica hash + firma JWT).

POST	/api/auth/registro
Rol requerido	admin
Para qué sirve	Da de alta un usuario (admin o conductor). Solo un admin puede hacerlo (evita escalada de privilegios).
Entrada (inputs)	{ "correo": "conductor@siol.com", "contrasena": "clave123", "rol": "conductor" }
Respuesta	{ "id": 2, "codigo": "CO-002", "correo": "conductor@siol.com", "rol": "conductor", "estado": true }
Se conecta con	usuario_service → usuario_repository; el rol se valida con un Enum (admin/conductor).

6.2 Clientes corporativos (/api/clientes)

GET	/api/clientes/
Rol requerido	admin
Para qué sirve	Lista las empresas cliente (las que envían los paquetes).
Entrada (inputs)	Ninguno.
Respuesta	[{ "id": 1, "codigo": "CL-001", "razon_social": "ACME SAC", "identificador_unico": "20100000001", "contacto": null }]
Se conecta con	cliente_service → cliente_repository → tabla clientes_corporativos.

POST	/api/clientes/
Rol requerido	admin
Para qué sirve	Registra una empresa cliente. (También se crean solas al cargar el Excel de pedidos.)
Entrada (inputs)	<pre>{ "razon_social": "ACME SAC", "identificador_unico": "20100000001", "contacto": "ventas@acme.com" }</pre>
Respuesta	El cliente creado (con su código CL-NNN).
Se conecta con	cliente_service (rechaza RUC duplicado) → cliente_repository.

6.3 Flota de vehículos (/api/vehiculos)

GET	/api/vehiculos/
Rol requerido	admin
Para qué sirve	Lista la flota de vehículos.
Entrada (inputs)	Ninguno.
Respuesta	<pre>[{ "id": 1, "codigo": "VE-001", "placa": "ABC-123", "marca": "Toyota", "capacidad_volumetrica": 5.0, "estado": "DISPONIBLE", "conductor_id": 2 }]</pre>
Se conecta con	vehiculo_service → vehiculo_repository → tabla vehiculos.

POST	/api/vehiculos/
Rol requerido	admin
Para qué sirve	Registra un vehículo. Si conductor_id es null, el vehículo es de la empresa; si no, es del conductor.
Entrada (inputs)	<pre>{ "placa": "ABC-123", "marca": "Toyota", "capacidad_volumetrica": 5.0, "estado": "DISPONIBLE", "conductor_id": 2 }</pre>
Respuesta	El vehículo creado (con su código VE-NNN).
Se conecta con	vehiculo_service (rechaza placa duplicada) → vehiculo_repository; conductor_id → tabla usuarios.

6.4 Gestión de pedidos / Inbound (/api/pedidos)

POST	/api/pedidos/upload
Rol requerido	admin
Para qué sirve	Carga masiva de pedidos desde un Excel (.xlsx). Crea/enlaza el cliente, guarda el destinatario, asigna el código PD-NNN y registra el primer evento de trazabilidad.
Entrada (inputs)	Archivo .xlsx (multipart). Columnas: direccion_destino (obligatoria) razon_social_cliente (obligatoria) ruc_cliente (opcional) nombre_destinatario (opcional) telefono_destinatario (opcional) dni_destinatario (opcional) referencia_externa (opcional, id del Excel) peso_kg, volumen_m3 (opcional)
Respuesta	<pre>{ "mensaje": "Carga masiva exitosa", "pedidos_nuevos": 5, "total_filas_leidas": 5 }</pre>
Se conecta con	pedido_service → pedido_repository + cliente_repository + historial_repository.

GET	/api/pedidos/
Rol requerido	admin
Para qué sirve	Lista los pedidos (paginado).
Entrada (inputs)	Query (opcional): skip=0 & limit=100
Respuesta	Lista de pedidos con todos sus campos (codigo PD-NNN, cliente, destinatario, estado, etc.).
Se conecta con	pedido_service → pedido_repository.

POST	/api/pedidos/geocodificar
Rol requerido	admin
Para qué sirve	Convierte las direcciones en coordenadas (lat/lng) y deduce el distrito. Tarda ~1 s por pedido.
Entrada (inputs)	Ninguno (procesa los pedidos sin coordenadas).
Respuesta	<pre>{ "mensaje": "Proceso ... finalizado", "pedidos_exitosos": 5, "pedidos_fallidos": 0 }</pre>
Se conecta con	pedido_service → geocoder (Geopy/Nominatim) → pedido_repository.

GET	/api/pedidos/zonas
Rol requerido	admin
Para qué sirve	Agrupar los pedidos geocodificados por distrito (para decidir las rutas).
Entrada (inputs)	Ninguno.
Respuesta	<pre>{ "zonas_operativas": [{ "distrito": "San Miguel", "total_pedidos": 3 }, { "distrito": "Miraflores", "total_pedidos": 2 }] }</pre>
Se conecta con	pedido_service → pedido_repository (agrupación SQL).

6.5 Enrutamiento (/api/rutas)

POST	/api/rutas/asignar-bloque
Rol requerido	admin
Para qué sirve	Crea una ruta para un conductor con TODOS los pedidos PENDIENTES de un distrito (CUS-18).
Entrada (inputs)	<pre>{ "nombre_ruta": "Ruta San Miguel - Mañana", "distrito": "San Miguel", "conductor_id": 2 }</pre>
Respuesta	<pre>{ "mensaje": "3 pedidos asignados...", "ruta_id": 1, "codigo": "RT-001" }</pre>
Se conecta con	ruta_service → ruta_repository + pedido_repository + historial_repository.

POST	/api/rutas/conductor/optimizar
Rol requerido	conductor
Para qué sirve	El conductor optimiza el ORDEN de entrega de SU ruta desde su posición actual (algoritmo del vecino más cercano). Verifica que la ruta sea suya.
Entrada (inputs)	<pre>{ "ruta_id": 1, "latitud_actual_conductor": -12.077, "longitud_actual_conductor": -77.083 }</pre>
Respuesta	<pre>{ "mensaje": "Ruta optimizada matemáticamente", "total_paradas": 3 }</pre>
Se conecta con	ruta_service → services/router (VRP) → ruta_repository + historial_repository.

6.6 App móvil del conductor (/api/conductor)

Todos exigen rol **conductor**. El conductor se identifica por su token (nunca por parámetro), así nunca puede ver la ruta de otro.

GET	/api/conductor/ruta-activa
Rol requerido	conductor
Para qué sirve	Devuelve el resumen de la ruta activa del conductor y su avance (CUS-21).
Entrada (inputs)	Ninguno (se deduce del token).
Respuesta	<pre>{ "ruta_id": 1, "codigo": "RT-001", "nombre": "...", "estado": "EN_PROGRESO", "total_paradas": 3, "pendientes": 1, "entregadas": 2, "fallidas": 0 }</pre>
Se conecta con	ruta_service → ruta_repository.

GET	/api/conductor/ruta-activa/manifiesto
Rol requerido	conductor
Para qué sirve	Lista detallada de paradas, ordenadas por secuencia, con datos del destinatario (CUS-24).
Entrada (inputs)	Ninguno.
Respuesta	<pre>{ "ruta_id": 1, "codigo": "RT-001", "paradas": [{ "secuencia": 1, "codigo": "PD-001", "cliente_origen": "ACME SAC", "nombre_destinatario": "Juan Perez", "direccion_destino": "...", "estado_entrega": "PENDIENTE" }] }</pre>
Se conecta con	ruta_service → ruta_repository.

GET	/api/conductor/ruta-activa/navegacion
Rol requerido	conductor
Para qué sirve	Waypoints (lat/lng) ordenados para alimentar el mapa de la app (CUS-25).
Entrada (inputs)	Ninguno.
Respuesta	<pre>{ "ruta_id": 1, "total_paradas": 3, "paradas": [{ "secuencia": 1, "codigo": "PD-001", "latitud": -12.07, "longitud": -77.08 }] }</pre>
Se conecta con	ruta_service → ruta_repository.

POST	/api/conductor/almacen/validar-qr
Rol requerido	conductor
Para qué sirve	Valida si el paquete escaneado (código PD-NNN del QR) pertenece a la ruta del conductor (CUS-22).
Entrada (inputs)	<pre>{ "codigo": "PD-001" }</pre>
Respuesta	<pre>{ "pertenece": true, "mensaje": "Paquete validado...", "parada": { ... } }</pre>
Se conecta con	ruta_service → ruta_repository (busca el pedido por su código).

PATCH	/api/conductor/paradas/{pedido_id}/estado
Rol requerido	conductor
Para qué sirve	Marca una entrega como ENTREGADO o FALLIDO. Si es FALLIDO, el motivo es obligatorio (CUS-26). Registra el evento en el historial.
Entrada (inputs)	Ruta: pedido_id en la URL <pre>{ "estado": "ENTREGADO" }</pre> // o: { "estado": "FALLIDO", // "motivo_fallo": "Cliente ausente" }
Respuesta	<pre>{ "pedido_id": 1, "codigo": "PD-001", "estado_entrega": "ENTREGADO", "fecha_gestion": "2026-06-06T15:00:00", "mensaje": "Pedido marcado como ENTREGADO" }</pre>
Se conecta con	ruta_service → ruta_repository + historial_repository.

POST	/api/conductor/paradas/{pedido_id}/evidencia
Rol requerido	conductor
Para qué sirve	Sube la foto de prueba de entrega (POD). Solo acepta imágenes (CUS-29).
Entrada (inputs)	Archivo de imagen (.jpg/.png/.webp) en multipart (campo "file").
Respuesta	<pre>{ "pedido_id": 1, "codigo": "PD-001", "url_evidencia": "/media/evidencias/pod_1_1.jpg", "mensaje": "Evidencia (POD) cargada correctamente" }</pre>
Se conecta con	ruta_service guarda el archivo en uploads/evidencias y la URL en el detalle. Se sirve en /media.

POST	/api/conductor/ruta-activa/finalizar
Rol requerido	conductor
Para qué sirve	Da por finalizada la ruta del día (CUS-28). Devuelve el resumen de entregas.
Entrada (inputs)	Ninguno.
Respuesta	<pre>{ "ruta_id": 1, "codigo": "RT-001", "estado": "FINALIZADA", "entregadas": 2, "fallidas": 1, "pendientes": 0, "mensaje": "Ruta finalizada correctamente" }</pre>
Se conecta con	ruta_service → ruta_repository.

6.7 Dashboard y trazabilidad (/api/dashboard)

GET	/api/dashboard/resumen
Rol requerido	admin
Para qué sirve	KPIs globales: total de pedidos por estado y conteo de rutas (CUS-33).
Entrada (inputs)	Ninguno.
Respuesta	<pre>{ "total_pedidos": 5, "pedidos_por_estado": {"ENTREGADO": 2, "PENDIENTE": 3}, "total_rutas": 1, "rutas_activas": 1, "rutas_finalizadas": 0 }</pre>
Se conecta con	dashboard_service → pedido_repository + ruta_repository.

GET	/api/dashboard/flota
Rol requerido	admin
Para qué sirve	Estado y avance (%) de todas las rutas, con el conductor asignado (CUS-33).
Entrada (inputs)	Ninguno.
Respuesta	<pre>{ "total_rutas": 1, "rutas": [{ "ruta_id": 1, "nombre": "...", "estado": "EN_PROGRESO", "conductor_correo": "c@siol.com", "total_paradas": 3, "entregadas": 2, "avance_porcentaje": 66.7 }] }</pre>
Se conecta con	dashboard_service → ruta_repository + usuario_repository.

GET	/api/dashboard/pedidos/{codigo}/historial
Rol requerido	admin
Para qué sirve	Línea de tiempo COMPLETA de un paquete por su código PD-NNN (CUS-35): cada cambio de estado, cuándo y quién lo hizo.
Entrada (inputs)	Ruta: el código del pedido en la URL (ej. PD-001)
Respuesta	<pre>{ "codigo": "PD-001", "estado_actual": "ENTREGADO", "ruta_asignada": "RT-001", "eventos": [{"evento": "PENDIENTE", "realizado_por": "admin@siol.com"}, {"evento": "ASIGNADO", ...}, {"evento": "EN_RUTA", "realizado_por": "c@siol.com"}, {"evento": "ENTREGADO", ...}] }</pre>
Se conecta con	dashboard_service → historial_repository (tabla historial_pedidos) + pedido/ruta.

7. Flujo completo de extremo a extremo

Este es el orden correcto de uso. Los datos que se generan (códigos) se usan en los pasos siguientes.

Lado ADMIN (panel web):

- Inicia sesión (admin) y autoriza en Swagger.
- (Opcional) Registra conductores y vehículos.
- POST /api/pedidos/upload — sube el Excel. Se crean clientes (CL-) y pedidos (PD-).
- POST /api/pedidos/geocodificar — calcula coordenadas y distritos.
- GET /api/pedidos/zonas — revisa cuántos pedidos hay por distrito.
- POST /api/rutas/asignar-bloque — crea la ruta (RT-) para un conductor.

Lado CONDUCTOR (app móvil):

- Inicia sesión (conductor) y autoriza.
- POST /api/rutas/conductor/optimizar — ordena su ruta.
- GET /api/conductor/ruta-activa, /manifiesto, /navegacion — descarga su trabajo.
- POST /api/conductor/almacen/validar-qr — valida cada paquete (PD-) en el almacén.
- PATCH /api/conductor/paradas/{id}/estado — marca ENTREGADO/FALLIDO.
- POST /api/conductor/paradas/{id}/evidencia — sube la foto POD.
- POST /api/conductor/ruta-activa/finalizar — cierra la ruta del día.

Lado ADMIN (monitoreo, en cualquier momento):

- GET /api/dashboard/resumen y /flota — ve el avance en vivo.
- GET /api/dashboard/pedidos/PD-001/historial — la trazabilidad completa de un paquete.

8. Seguridad

El backend implementa medidas alineadas con OWASP Top 10 y buenas prácticas.

8.1 Autenticación y mínimo privilegio

- **Contraseñas con Argon2** (hash moderno). Nunca se guardan en texto plano.
- **Token JWT** firmado (HS256), con expiración. Se envía como «Authorization: Bearer».
- **Control por rol**: cada endpoint exige admin o conductor (dependencias `get_current_admin` / `get_current_conductor`).
- **Verificación de propiedad**: un conductor solo accede a SU ruta, no a la de otros.
- **Registro cerrado**: solo un admin puede crear usuarios; el rol se valida con un Enum. Esto evita la escalada de privilegios (que cualquiera se cree admin).

8.2 Gestión de secretos (sin credenciales en el código)

- La clave del JWT, las credenciales de la BD y el CORS se leen de **variables de entorno** (módulo `app/core/config.py`).
- El archivo `.env` está en `.gitignore` (no se sube al repositorio). Hay una plantilla `.env.example`.

8.3 Configuración y comunicación

- **CORS configurable** por entorno (no abierto a todo en producción).
- **Sin inyección SQL**: todo el acceso a datos usa el ORM (consultas parametrizadas).
- **Validación de archivos**: el Excel valida extensión; la evidencia solo acepta imágenes.

8.4 Trazabilidad / auditoría

La tabla **historial_pedidos** registra cada cambio de estado con su fecha y el usuario que lo hizo. Es un rastro de auditoría de todas las operaciones.

8.5 Gestión de vulnerabilidades y CI/CD

- **Dependencias con versión fija** (`requirements.txt`) para poder rastrear CVEs.
- **GitHub Actions** (`.github/workflows/ci.yml`): en cada push corre pruebas (pytest), análisis estático de seguridad (bandit) y auditoría de dependencias (pip-audit).
- **Dependabot** (`.github/dependabot.yml`): abre PRs semanales con actualizaciones de seguridad.
- **Secret Scanning**: se activa en GitHub (Settings → Code security) para detectar secretos filtrados.

8.6 Cumplimiento OWASP Top 10 (resumen)

A01 Control de acceso	Cubierto	RBAC + propiedad + registro solo-admin
A02 Fallos criptográficos	Cubierto	Argon2 + secretos en entorno
A03 Inyección	Cubierto	ORM parametrizado
A05 Mala configuración	Parcial	CORS por entorno (faltan headers/TLS en despliegue)
A06 Componentes vulnerables	Cubierto	Pin + Dependabot + pip-audit
A07 Fallos de autenticación	Parcial	JWT + Argon2 (falta rate-limit)
A09 Registro y monitoreo	Parcial	historial_pedidos como auditoría

Pendiente para producción

- Proteger las fotos /media tras autenticación (hoy públicas, para simplificar el frontend).

- Rate-limiting en el login (anti fuerza bruta) y límite de tamaño de archivos.
- Cabeceras de seguridad (HSTS, etc.) y HTTPS/TLS en el despliegue.
- Política de contraseñas y refresh tokens.

9. Apéndice: mapa de archivos del proyecto

```
app/
  main.py          # arranque: crea app, tablas, admin inicial, routers
  core/
    config.py      # configuración desde variables de entorno (secretos)
    security.py    # hash de contraseñas (Argon2) + JWT
    codigos.py     # generación de códigos legibles (PD-001...)
  db/database.py   # conexión y sesión de PostgreSQL
  api/             # routers (entrada HTTP)
    auth, clientes, vehiculos, pedidos, rutas, conductor, dashboard, deps
  services/        # lógica de negocio
  repositories/    # acceso a datos (queries)
  models/          # tablas SQLAlchemy
  schemas/         # moldes Pydantic (validación de entrada/salida)
alembic/           # migraciones de la base de datos
.github/workflows/ci.yml # CI/CD (pruebas + seguridad)
.github/dependabot.yml # actualizaciones de dependencias
tests/            # pruebas automáticas (pytest)
docker-compose.yml # API + base de datos
.env.example      # plantilla de variables de entorno
SEGURIDAD.md      # detalle de seguridad
GUIA_PRUEBAS.md   # guía paso a paso para probar
```

Fin del manual — SIOL-SAVA Backend v1.0.0